WEEK 1 · LESSON 3 OF 24 · HANDOUT

Loss Functions and Gradient Descent

Study time: **about 50 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=IHZwWFHWa-w — open the lesson online to watch inside the course.

🎓 **Go deeper:** Andrej Karpathy: Neural networks and backpropagation, step by step (building micrograd) — https://www.youtube.com/watch?v=VMj-3S1tku0

🎓 **Go deeper:** MIT 6.S191: Introduction to Deep Learning (2024) — https://www.youtube.com/watch?v=ErnWZxJovaM

Your goal in this lesson: Measure how wrong a network is with mean squared error and cross-entropy, and train a single neuron with gradient descent.

A score for being wrong

To improve, a network needs one number that says how wrong it is: the **loss**. Training means changing the weights to make the loss smaller.

- **Mean squared error (MSE)** — for numbers: the average of (prediction − truth)². You used it in Math & Python for AI.
- **Cross-entropy** — for classes: punishes confident wrong answers very hard.

> *binary cross-entropy = −[y × log(p) + (1 − y) × log(1 − p)]*

where y is the true label (0 or 1) and p is the predicted probability of class 1.

```
import numpy as np

def bce(y, p):
    return -(y * np.log(p) + (1 - y) * np.log(1 - p))

for p in [0.99, 0.9, 0.6, 0.4, 0.1, 0.01]:
    print(f"true label 1, predicted {p:4.2f} -> loss {bce(1, p):.3f}")
```

A confident correct answer (0.99) costs almost nothing (about 0.01). A confident wrong answer (0.01) costs about **4.6**. Cross-entropy teaches the network not to be confidently wrong. For several classes, **categorical cross-entropy** is −log(probability given to the true class).

Gradient descent, again

The loss depends on every weight. The **gradient** tells us, for each weight, which direction makes the loss go **up** — so we step the other way:

> *new weight = old weight − learning rate × gradient*

This is exactly the gradient descent you did by hand for linear regression. Deep learning uses the same idea for millions of weights at once.

Train one neuron (logistic regression)

A single neuron with a sigmoid is logistic regression. For cross-entropy loss, the gradient has a beautifully simple form: (prediction − truth) × input.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split

X, y = make_moons(n_samples=1000, noise=0.2, random_state=0)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)
print(X_train.shape, X_test.shape)
plt.scatter(X[:, 0], X[:, 1], c=y, s=8)
plt.title("Two moons: a pattern a straight line cannot separate")
plt.show()
```

```
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

w = np.zeros(2)
b = 0.0
lr = 0.5
history = []
for epoch in range(500):
    p = sigmoid(X_train @ w + b)
    loss = bce(y_train, np.clip(p, 1e-9, 1 - 1e-9)).mean()
    history.append(loss)
    grad_w = X_train.T @ (p - y_train) / len(y_train)
    grad_b = (p - y_train).mean()
    w -= lr * grad_w
    b -= lr * grad_b

acc = ((sigmoid(X_test @ w + b) > 0.5) == y_test).mean()
print("final loss:", round(history[-1], 3), " test accuracy:", acc)
plt.plot(history)
plt.xlabel("epoch")
plt.ylabel("cross-entropy loss")
plt.show()
```

The loss falls quickly then levels off, and the single neuron reaches about **0.86** test accuracy. It cannot do better: one neuron draws one **straight** line, and the moons are curved. Next lesson, a hidden layer fixes that.

Check a gradient by nudging

You can always check a gradient numerically: nudge a weight a tiny bit and see how the loss changes.

```
def mean_loss(w, b):
    p = np.clip(sigmoid(X_train @ w + b), 1e-9, 1 - 1e-9)
    return bce(y_train, p).mean()

eps = 1e-5
w_plus = w.copy(); w_plus[0] += eps
w_minus = w.copy(); w_minus[0] -= eps
numeric = (mean_loss(w_plus, b) - mean_loss(w_minus, b)) / (2 * eps)
analytic = X_train[:, 0] @ (sigmoid(X_train @ w + b) - y_train) / len(y_train)
print(round(numeric, 6), round(analytic, 6))
```

The two numbers agree — our formula is right. This **gradient check** is a classic debugging trick.

Mini-batches and epochs

Real datasets are too big to use all at once. **Stochastic / mini-batch gradient descent** uses a small random batch (for example 32 or 64 examples) for each step. One pass through all the batches is an **epoch**. Batches make training faster and the noise even helps escape poor solutions.

Key terms

- **Loss** — one number measuring how wrong the predictions are.
- **Cross-entropy** — the standard loss for classification.
- **Gradient** — the direction in which each weight increases the loss.
- **Learning rate** — the size of each step.
- **Mini-batch** — a small group of examples used for one update.
- **Epoch** — one full pass through the training data.

Remember this: Training is a loop — predict, measure the loss, compute gradients, step downhill — and cross-entropy is the loss for classification.

Name: _____ Date: _____

Activity

Task 1

Try learning rates 0.05, 0.5 and 5. Plot the three loss curves on one chart.

Task 2

Change the training loop to use random mini-batches of 32 rows. Does it still learn?

Task 3

Compute the categorical cross-entropy when the true class gets probability 0.7, then 0.05.

Task 4

Plot the straight decision line $w_1x_1 + w_2x_2 + b = 0$ on top of the moons scatter plot.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What does the loss measure?
   - A. The number of layers
   - B. The training speed
   - C. The accuracy on new data only
   - D. How wrong the predictions are
2. Which loss is standard for classification?
   - A. R^2
   - B. Cross-entropy
   - C. Median error
   - D. Mean squared error
3. The true label is 1 and the model predicts 0.01. What happens to the cross-entropy loss?
   - A. It is almost 0
   - B. It is large (about 4.6)
   - C. It is negative
   - D. It is exactly 1
4. In gradient descent, which way does each weight move?
   - A. In the direction of the gradient
   - B. It does not move
   - C. At random
   - D. Opposite to the gradient, downhill
5. Why can a single neuron not reach high accuracy on the two moons?
   - A. It can only draw a straight boundary
   - B. It has no bias
   - C. The learning rate is wrong
   - D. The data has no labels
6. What is one epoch?
   - A. One test prediction
   - B. One full pass through the training data
   - C. One layer
   - D. One weight update

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.